

Yesterday 9:36 PM

✦ Upgrade

📤 Share

...

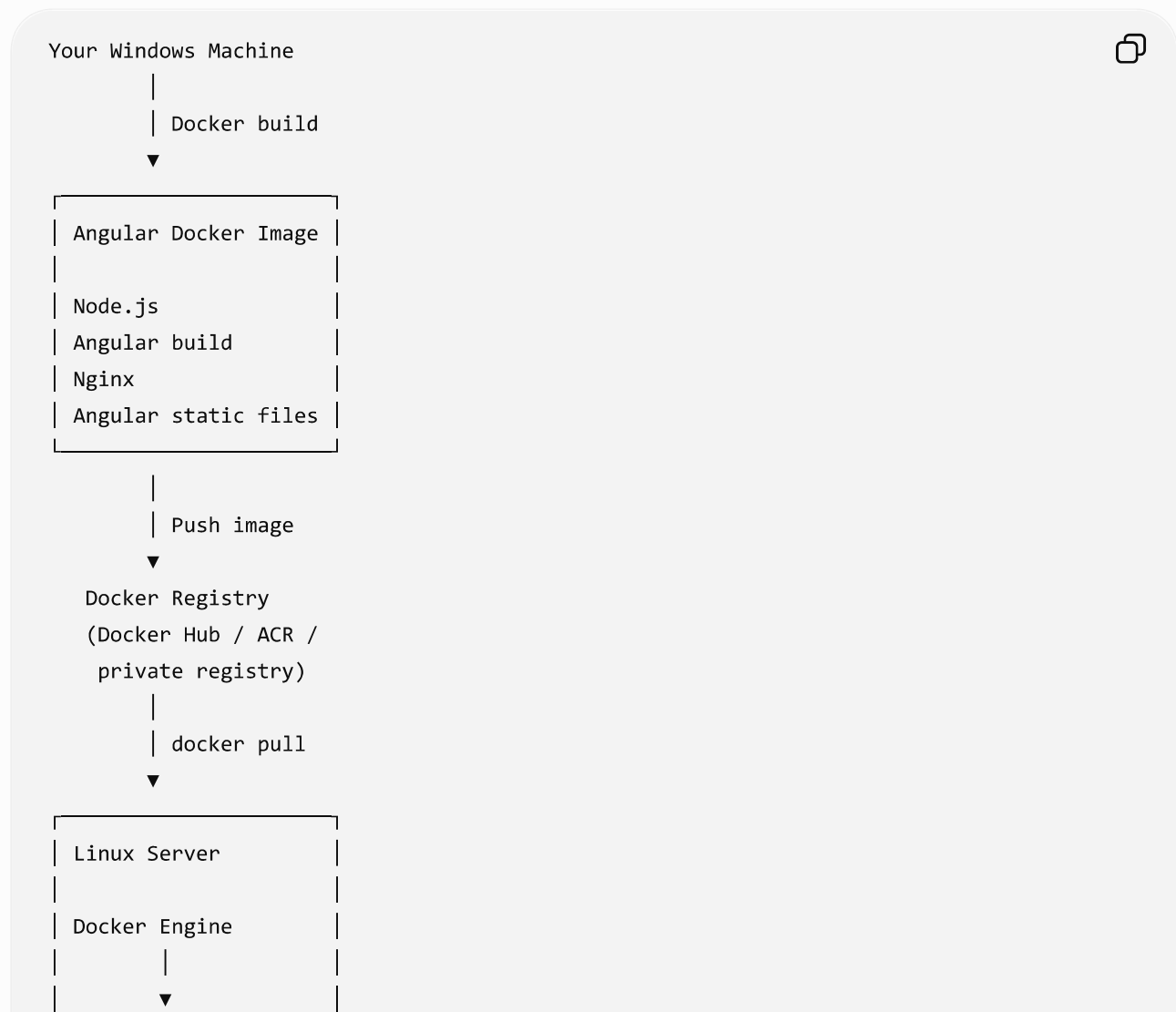
suppose i created a docker angular application in my windows machine, then will run that application in another linux machine using the docker file. is it possible ?

Yes — **absolutely possible**. This is one of the main benefits of Docker.

If you build your Angular application inside a Docker image on your **Windows machine**, you can run that same image on a **Linux machine**, provided the target Linux machine has Docker installed.

The basic idea

Your flow would look like:



Why does this work?

Docker packages your application and its **runtime environment** into an image.

For example, your Angular application might have:

```
Angular Application
+
Node.js
+
Nginx
+
configuration
+
dependencies
↓
Docker Image
```



The Linux machine doesn't need to install:

- Node.js
- Angular CLI
- npm dependencies
- Nginx manually

It primarily needs:

```
Linux
+
Docker
↓
Run your Docker image
```



Example

On Windows:

```
</> Bash
```

```
docker build -t my-angular-app:1.0 .
```



Then test it:

</> Bash



```
docker run -p 8080:80 my-angular-app:1.0
```

You can push the image to a registry:

</> Bash



```
docker tag my-angular-app:1.0 yourusername/my-angular-app:1.0
```

```
docker push yourusername/my-angular-app:1.0
```

On the Linux server:

</> Bash



```
docker pull yourusername/my-angular-app:1.0
```

Then:

</> Bash



```
docker run -d -p 80:80 yourusername/my-angular-app:1.0
```

Now users can access:

`http://<linux-server-ip>`



One important distinction

There are actually **two things** involved:

1. Docker image portability

This is highly portable:

Windows development machine



Docker Image



Linux server

2. CPU architecture

You need to consider the processor architecture.

For example:

```
Windows x86-64
  ↓
Linux x86-64
```



Usually straightforward.

But:

```
Windows x86-64
  ↓
Linux ARM64
```



may require a multi-platform image.

You can build one using:

```
</> Bash

docker buildx build \
  --platform linux/amd64,linux/arm64 \
  -t yourusername/my-angular-app:1.0 \
  --push .
```



For a normal **Windows development machine** → **x86-64 Linux server**, you generally don't need to worry much about this.

And there's an important Angular detail

For an Angular production application, you normally **don't need Node.js running in production** if you're using the traditional static Angular deployment.

A common Dockerfile is a **multi-stage build**:

```
</> dockerfile

# Stage 1: Build Angular
FROM node:22-alpine AS build

WORKDIR /app

COPY package*.json ./
```



```
RUN npm ci
```

```
COPY . .
```

```
RUN npm run build
```

```
# Stage 2: Serve Angular
```

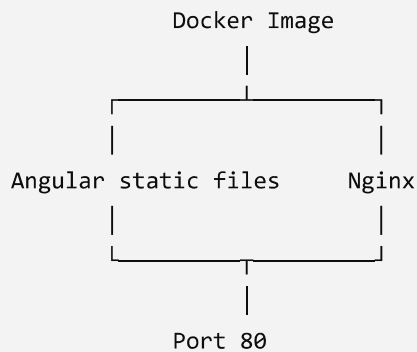
```
FROM nginx:alpine
```

```
COPY --from=build /app/dist/my-angular-app/browser /usr/share/nginx/html
```

```
EXPOSE 80
```

```
CMD ["nginx", "-g", "daemon off;"]
```

So the final production container is essentially:



That's a very common production approach.

The key concept to remember

Docker doesn't mean "Windows application running on Linux."

Rather:

You build a container image containing your application and its Linux-based runtime environment, then Docker runs that environment on the Linux host.

So your development environment can be:

Windows + Docker Desktop

while production can be:

Linux + Docker Engine

and the **same Docker image** can be used on both.

One caveat: if your Angular app is **SSR/hybrid rendering**, the Docker setup is slightly different because Node.js may need to remain in the production container.



so that's mean without installing nodejs, npm in linux machine, i can run the docker application but i need to install docker in the linux machine. is it correct ?

Yes — exactly correct. 👍

For your Angular Docker application, the Linux machine **does not need Node.js or npm installed on the host**.

It needs:



```
Linux Server
|
├─ Docker Engine      ✓ Required
|
├─ Node.js            ✗ Not required on host
├─ npm                ✗ Not required on host
├─ Angular CLI        ✗ Not required on host
└─ Angular application ✗ Not installed directly
```

The Docker image contains whatever your application needs.

Example

If your Dockerfile has:



```
</> dockerfile

FROM node:22-alpine AS build

WORKDIR /app

COPY package*.json ./
RUN npm ci
```

```
COPY . .  
RUN npm run build
```

`node` and `npm` exist inside the **Docker build environment**, not on the Linux host.

Then if you use:

```
</> dockerfile  
  
FROM nginx:alpine
```

your final production container can contain:

```
Linux Server  
├──  
└── Docker  
    ├──  
    └── Angular Container  
        ├── Nginx  
        └── Angular dist files
```

So the Linux server doesn't need:

```
</> Bash  
  
node --version  
npm --version  
ng version
```

to work.

It only needs Docker:

```
</> Bash  
  
docker --version
```

Then:

```
</> Bash  
  
docker pull your-image  
docker run -d -p 80:80 your-image
```

One important distinction

If you **build the Docker image on your Windows machine** and then transfer the image to Linux, Linux doesn't even need Node.js/npm during the build.



That's one of the core ideas behind Docker: package the application and its environment so the host machine doesn't need the application's development/runtime dependencies.

For your Angular 19 setup, this is also a good way to understand **why Docker is useful in CI/CD and production deployment**.



ChatGPT can make mistakes. Check important info.